

TP6 : SGBD Realm

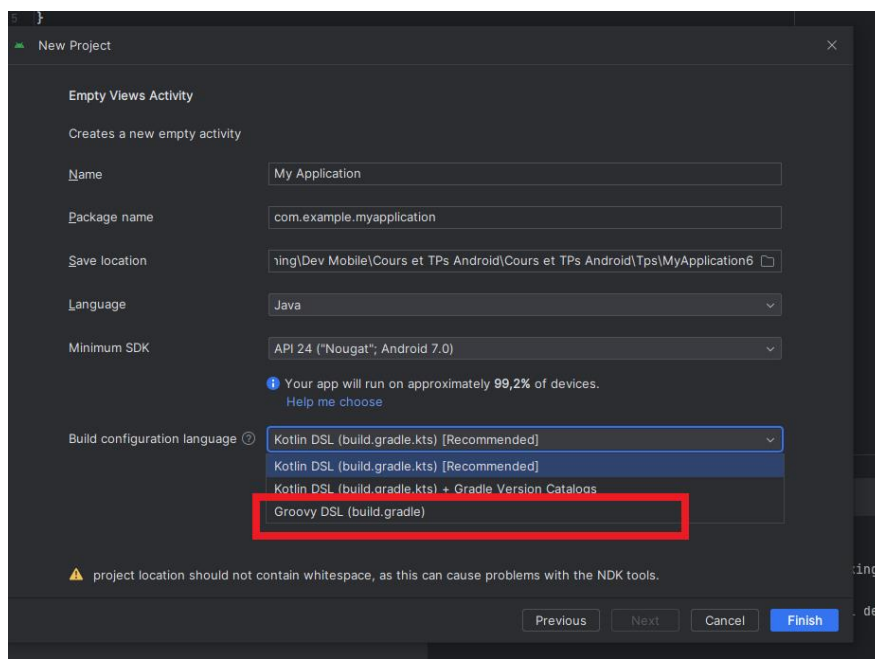
Realm est une base de données dans le cloud qui s'intègrent très facilement et qui est présentée comme une véritable base de données d'objets : Elle se différencie des autres en traitant les objets de données comme des objets vivants, ce qui signifie que les objets sont automatiquement mis à jour.

La particularité de Realm est d'être à la fois une solution de base de données en local sur le téléphone et dans le cloud. En effet, vous pouvez souscrire à leur offre de plateforme, ou alors vous pouvez utiliser leur technologie de base de données open source (donc gratuite) et utiliser Realm pour vos données en local.

L'objectif de ce TP est de se familiariser avec l'utilisation de la base de données Realm dans Android Studio. Cet exercice consiste à créer une application simple qui permet d'ajouter, de lire, de mettre à jour et de supprimer des données utilisateur.

Installation des outils :

Juste une petite précision, au moment de la création du projet pour ce TP il faut choisir **Groovy DSL** comme build configuration Language. Sinon le code que vous allez ajouter ton le fichier build.gradle ne va pas fonctionne (voir image ci-dessous).



Configuration du Realm:

Dans le fichier build.gradle du projet, pour ajouter Realm, vous devez éditer les deux fichiers build.gradle :

```
buildscript {
    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:8.3.1'
        classpath 'io.realm:realm-gradle-plugin:10.15.0'
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}
```

Configuration de l'application : Dans le fichier build.gradle du dossier app :

```
plugins {
    id 'com.android.application'
    id 'realm-android'
}

android {
    namespace 'com.example.myapplication'
    compileSdk 34
}
```

La configuration de l'environnement de construction de votre projet Gradle : gradle.properties :

- **org.gradle.warning.mode=all** : Cette option indique à Gradle de montrer tous les avertissements durant la construction
- **org.gradle.jvmargs=--add-opens jdk.compiler/com.sun.tools.javac.code=ALL-UNNAMED** : Cette option est utilisée pour configurer des arguments spécifiques de la JVM qui seront utilisés par Gradle.

```
org.gradle.warning.mode=all
org.gradle.jvmargs=--add-opens jdk.compiler/com.sun.tools.javac.code=ALL-UNNAMED
```

Initialiser Realm

Pour initialiser Realm correctement dans une application Android, il est préférable de le faire dès le démarrage de l'application. Cela se fait généralement dans une classe d'application personnalisée, qui est une sous-classe d'Application et est exécutée avant toute autre partie de votre code d'application Android :

1. Créer la classe EtudiantApplication, qui étend la classe Application, contenant des

propriétés globales, tel que un login et un password, et la méthode onCreate() pour initialiser Realm dans l'application.

```
public class EtudiantApplication extends Application {
    2 usages
    private String login ;
    2 usages
    private String password;

    public void onCreate() {
        super.onCreate();
        Realm.init( context: this);
    }
    no usages
    public void setLogin(String login) {
```

2. Préciser le nouveau contexte de l'application dans le fichier Manifest.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:name=".EtudiantApplication"
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.AppCompat" >
```

Définition du modèle de données :

Un modèle Realm est une classe qui étend RealmObject. Il permet de gérer l'objet et de fournir des fonctionnalités de persistance :

1. Créer la classe Etudiant disposant des informations suivantes : id, le nom_prenom et l'âge ;
2. Spécifier la clé primaire à l'aide de l'annotation @PrimaryKey ainsi que les getters et setters des propriétés.

```
public class Etudiant extends RealmObject {
    2 usages
    @PrimaryKey
    private long id;
    2 usages
    private String name;
    2 usages
    private int age;

    // Getters et setters
    1 usage
    public long getId() {
```

Création et Manipulation des données :

Pour manipuler efficacement les données dans une base de données Realm avec un modèle Etudiant, on va créer, lire, mettre à jour et supprimer des objets (les opérations CRUD (Create, Read, Update, Delete) du modèle Etudiant). Créer une classe nommée DataManager contenant les fonctionnalités suivantes :

➤ Ajouter un Etudiant :

```
public void addEtudiant(long id, String name, int age) {
    Realm realm = Realm.getDefaultInstance();
    realm.executeTransaction(r -> {
        Etudiant user = r.createObject(Etudiant.class, id);
        user.setName(name);
        user.setAge(age);
    });
}
```

- Récupérer tous les étudiants :

```
public List<Etudiant> getAllUsers() {  
    Realm realm = Realm.getDefaultInstance();  
    return realm.where(Etudiant.class).findAll();  
}
```

- Modifier un Etudiant :

```
public void updateUser(long id, String newName, int newAge) {  
    Realm realm = Realm.getDefaultInstance();  
    realm.executeTransaction(r -> {  
        Etudiant user = r.where(Etudiant.class).equalTo("id", id).findFirst();  
        if (user != null) {  
            user.setName(newName);  
            user.setAge(newAge);  
        }  
    });  
}
```

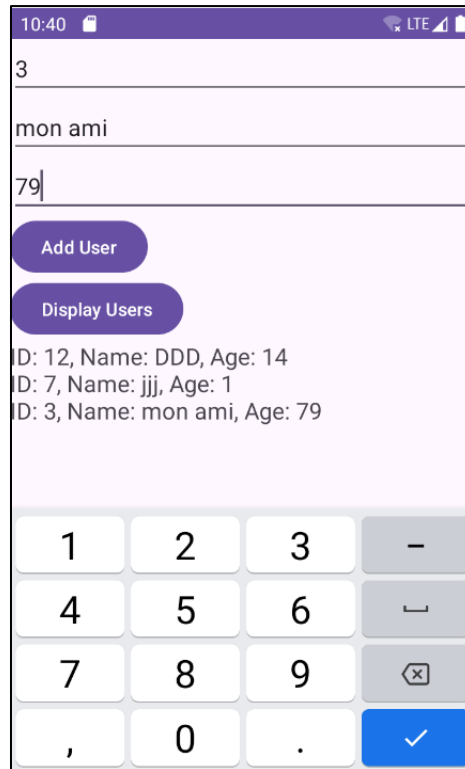
- Supprimer un Etudiant :

```
public void deleteUser(long id) {  
    Realm realm = Realm.getDefaultInstance();  
    realm.executeTransaction(r -> {  
        Etudiant user = r.where(Etudiant.class).equalTo("id", id).findFirst();  
        if (user != null) {  
            user.deleteFromRealm();  
        }  
    });  
}
```

Création de l'interface utilisateur :

Nous allons configurer notre activité principale qui comprendra une interface utilisateur permettant d'ajouter, lire, mettre à jour, et supprimer des utilisateurs. Voici les détails de la mise en œuvre :

- Implémenter l'interface suivante permettant d'ajouter et de lister tous les utilisateurs de la base de données :



➤ Implémenter l'activité principale :

1. Récupérer et manipuler les différents éléments de l'interface
2. Initialiser et récupérer l'instance de realm

```
public class MainActivity extends AppCompatActivity {
    2 usages
    private EditText userIdInput, userNameInput, userAgeInput;
    2 usages
    private TextView userDisplay;
    4 usages
    private Realm realm;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Initialize Realm
        Realm.init(context: this);
        realm = Realm.getDefaultInstance();

        userIdInput = findViewById(R.id.userId);
        userNameInput = findViewById(R.id.userName);
        userAgeInput = findViewById(R.id.userAge);
        userDisplay = findViewById(R.id.userDisplay);
        Button addUserButton = findViewById(R.id.addUserButton);
        Button readUsersButton = findViewById(R.id.readUsersButton);
    }
}
```

3. Implémenter la fonctionnalité d'ajout d'un étudiant :

```
addUserButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        addUser();
    }
});
```

```
private void addUser() {
    final long id = Long.parseLong(userIdInput.getText().toString());
    final String name = userNameInput.getText().toString();
    final int age = Integer.parseInt(userAgeInput.getText().toString());

    // Utiliser une transaction asynchrone
    realm.executeTransactionAsync(realm -> {
        Etudiant user = realm.createObject(Etudiant.class, id);
        user.setName(name);
        user.setAge(age);
    }, new Realm.Transaction.OnSuccess() {
        @Override
        public void onSuccess() {
            // La transaction a réussi.
            Log.d("tag: \"Realm\", msg: \"Utilisateur ajouté avec succès\");
        }
    }, new Realm.Transaction.OnError() {
        @Override
        public void onError(Throwable error) {
            // La transaction a échoué et a été automatiquement annulée.
            Log.e("tag: \"Realm\", msg: \"Échec de l'ajout de l'utilisateur\", error);
        }
    });
}
```

4. Modifier Le log.e par un Toast.

5. Implémenter la fonction d’affichage de tous les étudiants :

```
readUsersButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        displayUsers();
    }
});
```

```
private void displayUsers() {
    RealmResults<Etudiant> users = realm.where(Etudiant.class).findAll();
    StringBuilder builder = new StringBuilder();
    for (Etudiant user : users) {
        builder.append("ID: ").append(user.getId()).append(", Name: ").append(user.getName()).
            append(", Age: ").append(user.getAge()).append("\n");
    }
    userDisplay.setText(builder.toString());
}
```

6. Ajouter la fonction onDestroy pour fermer la base realm.

```
@Override
protected void onDestroy() {
    super.onDestroy();
    realm.close(); // Don't forget to close the Realm instance
}
```

7. Implémenter les fonctionnalités de modifications et de suppression